



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3
по курсу «Разработка параллельных и
распределенных программ»

*Параллельная реализация решения
системы линейных алгебраических уравнений с помощью
OpenMP*

Студент: Федуков А. А.

Группа: ИУ9-52Б

Преподаватель: Царев А. С.

Москва, 2025 г.

Содержание

Постановка задачи	2
РЕШЕНИЕ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВ- НЕНИЙ ИТЕРАЦИОННЫМИ МЕТОДАМИ	2
Практическая реализация	2
Код программы	3
Вывод программы	8
Вывод	9
Характеристики компьютера	9

Постановка задачи

РЕШЕНИЕ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ ИТЕРАЦИОННЫМИ МЕТОДАМИ

Пусть дана система из N линейных алгебраических уравнений в виде $Ax = b$, где A — матрица коэффициентов размером $N \times N$, b — вектор правых частей размером N , x — искомый вектор решений размером N .

Решение системы уравнений итерационным методом состоит в выполнении следующих шагов:

1. Задаётся $x^{(0)}$ — произвольное начальное приближение решения (вектор с произвольными начальными значениями).
2. Приближение многократно улучшается с использованием формулы вида $x^{(n+1)} = f(x^{(n)})$, где функция f определяется используемым методом¹.
3. Процесс продолжается, пока не выполнится условие $g(x^{(n)}) < \varepsilon$, где функция g определяется используемым методом, а величина ε задаёт требуемую точность.

¹Например, методом Якоби, методом Зейделя и др.

Практическая реализация

Код программы

Листинг 1: Исходный код программы

```
1 #include <cmath>
2 #include <iostream>
3 #include <vector>
4 #include <limits>
5 #include <omp.h>
6 #include <random>
7
8 constexpr int N = 10000;           // размерность матрицы
9 constexpr double EPS = 1e-4;       // критерий останова
10 constexpr double TAU = 0.1 / N;    // шаг итерации
11 constexpr int MAX_ITERS = 2'000'000; // максимум итераций
12
13 // 0 = не печатать, >0 = печатать каждые PRINT_EVERY итераций
14 constexpr int PRINT_EVERY = 0;
15 // =====
16
17 // разбиение строк [0..N) на p кусков, кусок tid: [row_start, row_end)
18 static inline void get_rows_range(int N, int p, int tid, int &row_start,
19     int &row_end)
20 {
21     int base = N / p;
22     int extra = N % p;
23
24     if (tid < extra)
25     {
26         row_start = tid * (base + 1);
27         row_end = row_start + (base + 1);
28     }
29     else
30     {
31         row_start = extra * (base + 1) + (tid - extra) * base;
32         row_end = row_start + base;
33     }
34 }
35
36 int main()
37 {
38     std::cout << "OpenMP threads = " << omp_get_max_threads() << "\n";
```

```

39 // x_true = 1 (решение системы)
40 std::vector<double> x_true(N, 1.0);
41
42 // Матрица A (NxN) в 1D (row-major): A[i*N + j]
43 std::vector<double> A;
44 try
45 {
46     // Пытаемся выделить память под A
47     A.assign(static_cast<size_t>(N) * static_cast<size_t>(N), 0.0);
48 }
49 catch (const std::bad_alloc &)
50 {
51     std::cerr << "Not enough RAM to allocate A (" << N << "x" << N
52     << ").\n";
53     return 1;
54 }
55 // Заполняем A параллельно
56 // по диагонали 2, вне диагонали 1
57 #pragma omp parallel
58 {
59     int tid = omp_get_thread_num(); // номер текущего потока
60     int nthr = omp_get_num_threads(); // всего потоков
61     int rs = 0, re = 0;
62     get_rows_range(N, nthr, tid, rs, re); // получаем диапазон строк
        для этого потока
63
64     // Заполняем строки с rs до re
65     for (int i = rs; i < re; ++i)
66     {
67         size_t row_off = static_cast<size_t>(i) * static_cast<size_t>
68         >(N);
69         for (int j = 0; j < N; ++j)
70         {
71             A[row_off + j] = (i == j ? 2.0 : 1.0);
72         }
73     }
74
75     // b = A * x_true
76     std::vector<double> b(N, 0.0);
77
78 // Заполняем вектор b так, чтобы решение системы было x_true (все единиц
79 // )
80 #pragma omp parallel
81 {

```

```

81         int tid = omp_get_thread_num();
82         int nthr = omp_get_num_threads();
83         int rs = 0, re = 0;
84         get_rows_range(N, nthr, tid, rs, re);
85
86         for (int i = rs; i < re; ++i)
87         {
88             const size_t row_off = static_cast<size_t>(i) * static_cast<
size_t>(N);
89             double sum = 0.0;
90             for (int j = 0; j < N; ++j)
91             {
92                 sum += A[row_off + j] * x_true[j];
93             }
94             b[i] = sum;
95         }
96     }
97
98     // norm_b = ||b||2
99     double norm_b2 = 0.0;
100
101     // сворачивание суммы квадратов элементов b в norm_b2
102     // в каждом потоке своя локальная копия norm_b2, в конце они суммируются
103     // schedule(static) итерации делятся заранее и равномерно между потоками
104     #pragma omp parallel for reduction(+: norm_b2) schedule(static)
105         for (int i = 0; i < N; ++i)
106             norm_b2 += b[i] * b[i];
107     const double norm_b = std::sqrt(norm_b2);
108
109     // Начальное приближение x_0 = random в [-1, 1]
110     std::vector<double> x_n(N, 0.0);
111     std::vector<double> x_np1(N, 0.0);
112
113     // Рандом в диапазоне [-1, 1]
114     #pragma omp parallel
115     {
116         int tid = omp_get_thread_num();
117         // фиксированный сид
118         std::mt19937 rng(12345u + static_cast<unsigned>(tid));
119         std::uniform_real_distribution<double> dist(-1.0, 1.0);
120
121     #pragma omp for schedule(static)
122         for (int i = 0; i < N; ++i)
123         {
124             x_n[i] = dist(rng);
125         }

```

```

126     }
127
128     const double t0 = omp_get_wtime();
129
130     int iters_done = 0;
131     double last_ratio = std::numeric_limits<double>::infinity();
132
133     for (int iter = 0; iter < MAX_ITERS; ++iter)
134     {
135         double global_r2 = 0.0;
136
137         // Параллельно по строкам: считаем Ax, r_i, и x_{n+1}[i]
138         #pragma omp parallel reduction(+ : global_r2)
139         {
140             int tid = omp_get_thread_num();
141             int nthr = omp_get_num_threads();
142             int rs = 0, re = 0;
143             get_rows_range(N, nthr, tid, rs, re);
144
145             double local_r2 = 0.0;
146
147             for (int i = rs; i < re; ++i)
148             {
149                 const size_t row_off = static_cast<size_t>(i) *
static_cast<size_t>(N);
150
151                 // Вычисляем (Ax)_i
152                 double ax = 0.0;
153                 for (int j = 0; j < N; ++j)
154                 {
155                     ax += A[row_off + j] * x_n[j];
156                 }
157
158                 // Вычисляем r_i = (Ax)_i - b_i
159                 const double r_i = ax - b[i];
160                 // Вычисляем локальное ||r||^2
161                 local_r2 += r_i * r_i;
162
163                 // Обновляем x_{n+1}
164                 x_np1[i] = x_n[i] - TAU * r_i;
165             }
166             // Суммируем локальные ||r||^2 в глобальный
167             global_r2 += local_r2;
168         }
169         // конец параллельной области
170

```

```

171 // Вычисляем  $g(x_n) = ||r||_2 / ||b||_2$ 
172 last_ratio = std::sqrt(global_r2) / norm_b;
173 iters_done = iter + 1;
174
175 // Переходим к следующей итерации
176 x_n.swap(x_npl);
177
178 // печатаем прогресс
179 if constexpr (PRINT_EVERY > 0)
180 {
181     if (iters_done % PRINT_EVERY == 0)
182     {
183         std::cout << "iter " << iters_done << ", g(x_n)=" <<
last_ratio << "\n";
184     }
185 }
186
187 if (last_ratio < EPS)
188     break;
189 }
190
191 // получаем время окончания вычислений
192 const double t1 = omp_get_wtime();
193
194 // max_abs_error относительно x_true=1
195 double max_err = 0.0;
196 // какому потоку своя копия max_err, мы ее вычисляем параллельно,
197 // а после по ним считается общий max
198 #pragma omp parallel for reduction(max : max_err) schedule(static)
199 for (int i = 0; i < N; ++i)
200 {
201     // считаем погрешность для i-го элемента
202     const double e = std::abs(x_n[i] - 1.0);
203     if (e > max_err)
204         max_err = e;
205 }
206
207 std::cout << "N=" << N << " EPS=" << EPS << " TAU=" << TAU << "
MAX_ITERS=" << MAX_ITERS << "\n";
208 // !
209 std::cout << "iters=" << iters_done << " g(x_n)=" << last_ratio << "
time_sec=" << (t1 - t0) << "\n";
210 std::cout << "x[0..3]=" << x_n[0] << ", " << x_n[1] << ", " << x_n
[2] << "\n";
211 std::cout << "max_abs_error=" << max_err << "\n";
212 // выводить количество итераций за которое сошелся алгоритм

```

```

213     // взять другое значение x_start
214
215     return 0;
216 }

```

Вывод программы

Инициализируем начальное значение вектора $x^{(0)}$ случайными числами в диапазоне $[-1, 1]$ с фиксированным seed для генератора случайных чисел, чтобы обеспечить воспроизводимость результатов.

- Размер системы: $N = 16384$.
- Критерий остановки: $\varepsilon = 1 \times 10^{-7}$.
- Шаг итерации: $\tau = \frac{0.1}{N}$.
- Максимальное число итераций: 2000000.
- Используемые числа процессов: $p = 1, 2, 4, 8, 16$.
- `iters` — количество выполненных итераций до достижения критерия остановки.
- `g(x_n)` — значение критерия остановки на последней итерации.
- `time_sec` — время выполнения программы в секундах.
- `max_abs_error` — максимальная абсолютная ошибка между вычисленным решением и истинным решением (вектор из всех 1.0).

Листинг 2: Вывод программы

```

1 constexpr int N = 10000;           // размерность матрицы
2 constexpr double EPS = 1e-4;       // критерий остановки
3 constexpr double TAU = 0.1 / N;     // шаг итерации
4
5 > g++ -fopenmp lab3.cpp -o lab3.out
6 > OMP_NUM_THREADS=1 ./lab3.out
7 OpenMP threads = 1
8 N=10000 EPS=0.0001 TAU=1e-05 MAX_ITERS=2000000

```



```

9  iters=91 g(x_n)=9.55551e-05 time_sec=29.4153
10 x[0..3]=[1.78297, 0.265461, 0.0837309]
11 max_abs_error=1.00231
12 > OMP_NUM_THREADS=2 ./lab3.out
13 OpenMP threads = 2
14 N=10000 EPS=0.0001 TAU=1e-05 MAX_ITERS=2000000
15 iters=91 g(x_n)=9.53668e-05 time_sec=15.094
16 x[0..3]=[1.77595, 0.258435, 0.076705]
17 max_abs_error=1.00265
18 > OMP_NUM_THREADS=4 ./lab3.out
19 OpenMP threads = 4
20 N=10000 EPS=0.0001 TAU=1e-05 MAX_ITERS=2000000
21 iters=91 g(x_n)=9.55461e-05 time_sec=8.7579
22 x[0..3]=[1.77874, 0.261226, 0.079496]
23 max_abs_error=0.999197
24 > OMP_NUM_THREADS=8 ./lab3.out
25 OpenMP threads = 8
26 N=10000 EPS=0.0001 TAU=1e-05 MAX_ITERS=2000000
27 iters=91 g(x_n)=9.62019e-05 time_sec=7.24034
28 x[0..3]=[1.79019, 0.272673, 0.090943]
29 max_abs_error=1.00967
30 > OMP_NUM_THREADS=16 ./lab3.out
31 OpenMP threads = 16
32 N=10000 EPS=0.0001 TAU=1e-05 MAX_ITERS=2000000
33 iters=91 g(x_n)=9.63467e-05 time_sec=7.69807
34 x[0..3]=[1.7877, 0.270183, 0.0884527]
35 max_abs_error=1.00718

```

Вывод

В данной лабораторной работе была реализована параллельная версия итерационного метода решения системы линейных алгебраических уравнений с использованием OpenMP. Были проведены эксперименты с различным числом потоков, что позволило оценить влияние параллелизации на производительность алгоритма. Результаты показали значительное сокращение времени выполнения при увеличении числа потоков, что подтверждает эффективность использования параллельных вычислений для решения задач линейной алгебры.

Характеристики компьютера

Листинг 3: Характеристики компьютера

```
1 > inxi --cpu --memory
2 Memory:
3   System RAM: total: 16 GiB available: 14.8 GiB used: 10.48 GiB (70.8%)
4   Array-1: capacity: 64 GiB slots: 4 modules: 4 EC: None
5   Device-1: Channel-A DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
6   Device-2: Channel-B DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
7   Device-3: Channel-C DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
8   Device-4: Channel-D DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
9 CPU:
10  Info: 6-core model: AMD Ryzen 5 7640HS w/ Radeon 760M Graphics bits:
      64
11    type: MT MCP cache: L2: 6 MiB
12  Speed (MHz): avg: 1200 min/max: 400/5017 cores: 1: 1390 2: 1100 3:
      1100
13    4: 1100 5: 1100 6: 1100 7: 1396 8: 1100 9: 1432 10: 1100 11: 1389
      12: 1100
```